Cantor–Schröder–Bernstein Theorem

An End-to-End Meta-Math Walkthrough

Dibakar Sigdel[1]

[1]*Mindverse Computing LLC, Lynnwood, WA 98087, USA*

(Dated: October 2026)

Abstract

Start chains at the part of A not hit by g. Follow $g \circ f$ forward from those starts; use f on that chain region and reverse g everywhere else. The report presents the target theorem, declared imports, a dependency graph, five-component node contracts, a continuous proof, a worked example, and a hypothesis-repair audit. It is an authored mathematical exposition, not a deployed-agent transcript or a formal proof-assistant certificate.

LF-02 / Mathematical Logic and Set Theory / Report v1

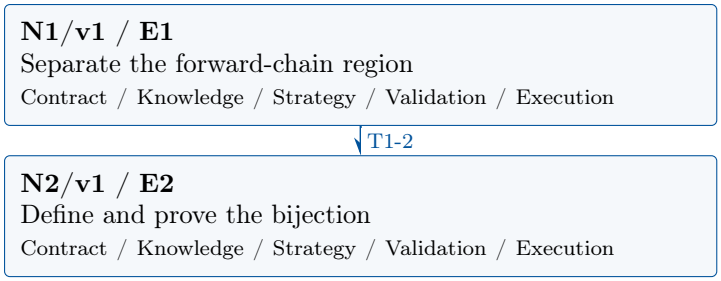
I. TARGET AND IMPORT CONTRACT

Theorem I.1. *If injections $f : A \to B$ and $g : B \to A$ exist, prove there is a bijection $h : A \to B$. The construction works for arbitrary sets and does not need a choice of preimages beyond the unique preimages already supplied by an injection.*

A. Foundations and permitted imports

Use ordinary set operations, recursion over $\mathbb{N}$, and injectivity. The notation $g^{-1}(a)$ below refers to the unique preimage of $a \in g(B)$; it is not defined on all of A. No cardinal arithmetic or axiom of choice is needed for the proof.

II. NODE INTERACTION WORKFLOW



Each box is one mathematical agent containing all five functional components. Blue arrows carry the versioned exports listed below. The layout gives a reading order; only drawn arrows assert dependencies. Diagnostic repair is separate from the proof DAG.

A. Versioned evidence-transfer ledger

T1-2: N1/E1/v1 to N2. $C \cap g(B) = g(f(C))$ and $A \setminus C \subset g(B)$.

Receiving activity. N2.1

Compatibility. E1/v1 and $h(a) = f(a)$ for $a \in C$, $h(a) = g^{-1}(a)$ for $a \notin C$. Check: Injectivity on each branch alone does not rule out collisions across branches.

III. AGENT ROLES AND FIVE-COMPONENT ACTIVITIES

A. N1/v1: Separate the forward-chain region

Object and role. Identify exactly which elements of the chain region lie in the image of g. **Local obligation:** $C \cap g(B) = g(f(C))$ and $A \setminus C \subset g(B)$.

Inputs. $A_0 = A \setminus g(B)$, $A_{n+1} = g(f(A_n))$, and $C = \bigcup_{n\geq 0} A_n$.

C — Contract

Publish E1/v1 only after establishing this obligation: $C \cap g(B) = g(f(C))$ and $A \setminus C \subset g(B)$.

K — Knowledge

$A_0 = A \setminus g(B)$, $A_{n+1} = g(f(A_n))$, and $C = \bigcup_{n\geq 0} A_n$. Retain the stated assumptions and import versions.

S — Strategy

Identify exactly which elements of the chain region lie in the image of g.

V — Validation

Every point outside C is in $g(B)$ because $A_0 \subset C$.

E — Execution

Carry out activities N1.1–N1.2 below, retaining each calculation and construction as evidence.

Numbered mathematical activities

N1.1 Every A_n with $n \geq 1$ lies in $g(B)$, whereas A_0 is disjoint from $g(B)$. Hence $C \cap g(B) = \bigcup_{n\geq 1} A_n$.

N1.2 Images commute with unions, so $g(f(C)) = \bigcup_{n\geq 0} g(f(A_n)) = \bigcup_{n\geq 1} A_n$. This proves the equality. Also if $a \notin C$, it cannot belong to $A_0 = A \setminus g(B)$, so it lies in $g(B)$.

Export E1/v1. $C \cap g(B) = g(f(C))$ and $A \setminus C \subset g(B)$.

Acceptance gate. Every point outside C is in $g(B)$ because $A_0 \subset C$.

Targeted repair. If g^{-1} is used outside its image, first prove the complement inclusion; it is what makes the second branch legitimate. Withhold this export and recheck its descendants after a versioned correction.

B. N2/v1: Define and prove the bijection

Object and role. Prove the two branch images are disjoint and together exhaust B. **Local obligation:** The piecewise-defined $h : A \to B$ is a bijection.

Inputs. E1/v1 and $h(a) = f(a)$ for $a \in C$, $h(a) = g^{-1}(a)$ for $a \notin C$. Incoming proof exports: E1/v1 at N2.1.

C — Contract

Publish E2/v1 only after establishing this obligation: The piecewise-defined $h : A \to B$ is a bijection.

K — Knowledge

E1/v1 and $h(a) = f(a)$ for $a \in C$, $h(a) = g^{-1}(a)$ for $a \notin C$. Incoming proof exports: E1/v1 at N2.1. Retain the stated assumptions and import versions.

S — Strategy

Prove the two branch images are disjoint and together exhaust B.

V — Validation

Injectivity on each branch alone does not rule out collisions across branches.

E — Execution

Carry out activities N2.1–N2.2 below, retaining each calculation and construction as evidence.

Numbered mathematical activities

N2.1 E1 makes h well defined. Each branch is injective. If $a \in C$ and $a' \notin C$ had $f(a) = g^{-1}(a')$, then $a' = g(f(a)) \in g(f(C)) \subset C$, a contradiction. Thus cross-branch collisions are impossible.

N2.2 For $b \in B$, if $b \in f(C)$ it is hit by the first branch. Otherwise $g(b) \notin C$: if it were in $C \cap g(B) = g(f(C))$, injectivity of g would force $b \in f(C)$. Thus $g(b) \in A \setminus C$, and the second branch maps it to b. This proves surjectivity and hence bijectivity.

Export E2/v1. The piecewise-defined $h : A \to B$ is a bijection.

Acceptance gate. Injectivity on each branch alone does not rule out collisions across branches.

Targeted repair. If surjectivity is asserted from the two original injections, explicitly split b according to membership in $f(C)$ and use E1 to identify its branch. Withhold this export and recheck its descendants after a versioned correction.

IV. CONTINUOUS MATHEMATICAL PROOF

A. N1 — Separate the forward-chain region

1. **N1.1.** Every A_n with $n \geq 1$ lies in $g(B)$, whereas A_0 is disjoint from $g(B)$. Hence $C \cap g(B) = \bigcup_{n \geq 1} A_n$.

2. **N1.2.** Images commute with unions, so $g(f(C)) = \bigcup_{n \geq 0} g(f(A_n)) = \bigcup_{n \geq 1} A_n$. This proves the equality. Also if $a \notin C$, it cannot belong to $A_0 = A \setminus g(B)$, so it lies in $g(B)$.

B. N2 — Define and prove the bijection

1. **N2.1.** E1 makes h well defined. Each branch is injective. If $a \in C$ and $a' \notin C$ had $f(a) = g^{-1}(a')$, then $a' = g(f(a)) \in g(f(C)) \subset C$, a contradiction. Thus cross-branch collisions are impossible.

2. **N2.2.** For $b \in B$, if $b \in f(C)$ it is hit by the first branch. Otherwise $g(b) \notin C$: if it were in $C \cap g(B) = g(f(C))$, injectivity of g would force $b \in f(C)$. Thus $g(b) \in A \setminus C$, and the second branch maps it to b. This proves surjectivity and hence bijectivity.

□

V. WORKED ORCHESTRATION

Take $A = B = \mathbb{N}_0$, $f(n) = n + 1$, and $g(n) = n + 1$. Then $A_0 = \{0\}$, $A_n = \{2n\}$, and C is the set of even numbers. The constructed bijection sends even n to $n + 1$ and odd n to $n - 1$, swapping every adjacent pair. This verifies both branches and their disjoint images.

A. Boundary and hypothesis audit

A piecewise choice of two injective rules can fail globally: on $\{0, 1\}$, assign $0 \mapsto 0$ on the first singleton branch and $1 \mapsto 0$ on the second. Each restricted map is injective, but the combined map is not. The cross-branch argument above is therefore an essential proof obligation.

VI. ILLUSTRATIVE FAILURE AND REPAIR

The following is an authored diagnostic scenario. Changed assumptions define a new contract; they do not retroactively prove the original statement.

A. Rejected argument, diagnosis, and correction

Rejected N2 proof: both pieces are injective, so h is injective. **Validation:** the two-singleton example isolates the missing cross-case. **Repair:** assume a cross collision and apply g to derive $a' \in g(f(C)) \subset C$, contradicting its branch membership. This fills the gap with no change of assumptions; recheck bijectivity after surjectivity is also established.

B. Publication and reuse

The corrected export retains its explicit assumptions. A hypothetical revision receives a new version, and each consuming activity is rechecked; this scenario is not a recorded runtime event. **Next direction:** Use explicit injections to compare infinite structures while retaining this construction as the justification for turning two inequalities into a bijection.

Provenance. This authored proof is complete relative to its declared imports. Reuse each export with its assumptions, proof, and version. The original Markdown remains available for comparison.

Further reading. Andrew Marks, Set Theory notes for sets and ultraproducts; Berkeley, Mathematical Logic for Mathematicians, Part I for first-order semantics; Andrew Marks, Computability Theory notes for effective procedures and undecidability. The proofs below specify their own set-theoretic and semantic assumptions.